Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation is a technique that combines information retrieval with text generation to produce more accurate and grounded responses. RAG systems first retrieve relevant documents from a knowledge base, then use a language model to synthesize an answer based on the retrieved context.

The key advantage of RAG over pure generation is that it can provide factual answers grounded in specific documents, reducing hallucination and enabling source attribution. This makes RAG particularly valuable for enterprise applications where accuracy and traceability are critical.

RAG architectures typically consist of three main components: an embedding model for converting text to vectors, a vector store for efficient similarity search, and a language model for generating responses. The embedding model creates dense vector representations of both documents and queries, enabling semantic search rather than simple keyword matching.

Vector Stores and Embeddings

Vector stores are specialized databases optimized for storing and querying high-dimensional vectors. Popular options include pgvector (PostgreSQL extension), Qdrant, Pinecone, and Weaviate. pgvector is particularly attractive for teams already using PostgreSQL, as it eliminates the need for a separate vector database service.

The embedding process converts text into fixed-dimensional vectors (typically 384 or 768 dimensions) that capture semantic meaning. Models like all-MiniLM-L6-v2 from Sentence Transformers are commonly used for this purpose. These models are trained on large text corpora and produce embeddings where semantically similar texts have high cosine similarity.

HNSW (Hierarchical Navigable Small World) graphs are the most popular indexing method for approximate nearest neighbor search in vector stores. They offer a good balance between query speed and recall accuracy.

Document Processing Pipeline

The document ingestion pipeline is responsible for converting raw documents (PDFs, Word files, PowerPoint presentations) into indexed chunks that can be searched. This process involves several steps: content extraction, text chunking, embedding generation, and vector storage.

Content extraction handles different file formats and produces structured text. For PDFs, tools like pdfplumber can extract text while preserving layout information. OCR may be needed for scanned documents. The extractor should also detect content types and element classifications (headings, tables, figures).

Text chunking splits long documents into smaller segments that fit within the embedding model context window. Fixed-size chunking with overlap is the simplest strategy, typically using 1000-character chunks with 200-character overlap. More advanced strategies consider semantic boundaries like paragraphs and sections.

Query Pipeline Architecture

The query pipeline orchestrates the full RAG flow: receiving a user question, embedding it, searching the vector store, filtering results, constructing a prompt with context, and calling the language model. Quality controls include relevance score thresholds, duplicate chunk detection, and token budget allocation.

When no relevant context is found (all chunks below the relevance threshold), the system should return a graceful message rather than hallucinating an answer. This is a critical quality control that prevents the LLM from generating unsupported claims.

The prompt template system uses composable Jinja2 templates for system instructions, context formatting, and conversation history. This allows operators to customize the RAG behavior without modifying application code. Template validation at startup ensures misconfiguration is caught early.

RAG Evaluation and Monitoring

Evaluating RAG systems requires measuring both retrieval quality and generation quality. Retrieval metrics include precision at k, recall, and mean reciprocal rank. Generation metrics include faithfulness (does the answer match the context), relevance (does the answer address the question), and factual accuracy.

A three-tier evaluation strategy works well in practice: synthetic test datasets in CI for regression detection, evaluation mode in staging with RAGAS or DeepEval frameworks, and production metrics collection for monitoring drift over time.

Observability in RAG systems goes beyond standard application monitoring. QueryTrace objects should capture per-step timing (embedding, search, filtering, LLM call), relevant metadata, and sufficient information for debugging quality issues without storing sensitive user content.